IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# Yenepoya Institute of Arts, Science, Commerce and Management

## Project – High Level Design

## on

# Identity Security Framework for E-Commerce

**Faya Yasmin**

**23BCCE106**

# Table of Contents


| | | |
|---|---|---|
| **1.** | **Introduction** | 1 |
| 1.1 | Scope of theDocument | 1 |
| 1.2 | Intended Audience | 1 |
| 1.3 | System Overview | 2 |
| **2.** | **System Design** | 3 |
| 2.1 | Application Design | 3 |
| 2.2 | Process Flow | 4 |
| 2.3 | Information Flow | 6 |
| 2.4 | Components Design | 6 |
| 2.5 | Key Design Considerations | 7 |
| 2.6 | API Catalogue | 8 |
| **3.** | **Data Design** | 9 |
| 3.1 | Data Model | 9 |
| 3.2 | Data Access Mechanism | 9 |
| 3.3 | Data Retention Policies | 10 |
| 3.4 | Data Migration | 10 |
| **4.** | **Interfaces** | 11 |
| **5.** | **State and Session Management** | 11 |
| **6.** | **Caching** | 12 |
| **7.** | **Non-Functional Requirements** | 12 |
| 7.1 | SecurityAspects | 12 |
| 7.2 | Performance Aspects | 12 |
| **8.** | **References** | 13 |

# 1. Introduction

The Identity Security Framework for E-Commerce Platform is a final-year academic project designed to demonstrate industry-standard cybersecurity practices within a working web application. The system addresses the fundamental challenge of digital identity management — ensuring that only authenticated and authorized users can access platform resources — using a technology stack representative of modern web development.

With e-commerce platforms handling sensitive personal data, financial information, and role-differentiated access, robust identity protection is a non-negotiable requirement. This project implements a multi-layered security model encompassing password hashing, email-based One-Time Password (OTP) verification, Role-Based Access Control (RBAC), and session lifecycle management.

## 1.1 Scope of the Document

This document covers the following aspects of the system:

Overall architectural design and layered component model
Functional modules and their interaction patterns
Authentication and authorization workflows
Database schema and data access design
Security controls and non-functional requirements
Deployment architecture and interface definitions
Assumptions, constraints, risks, and future enhancement roadmap

## 1.2 Intended Audience

This document is intended for:

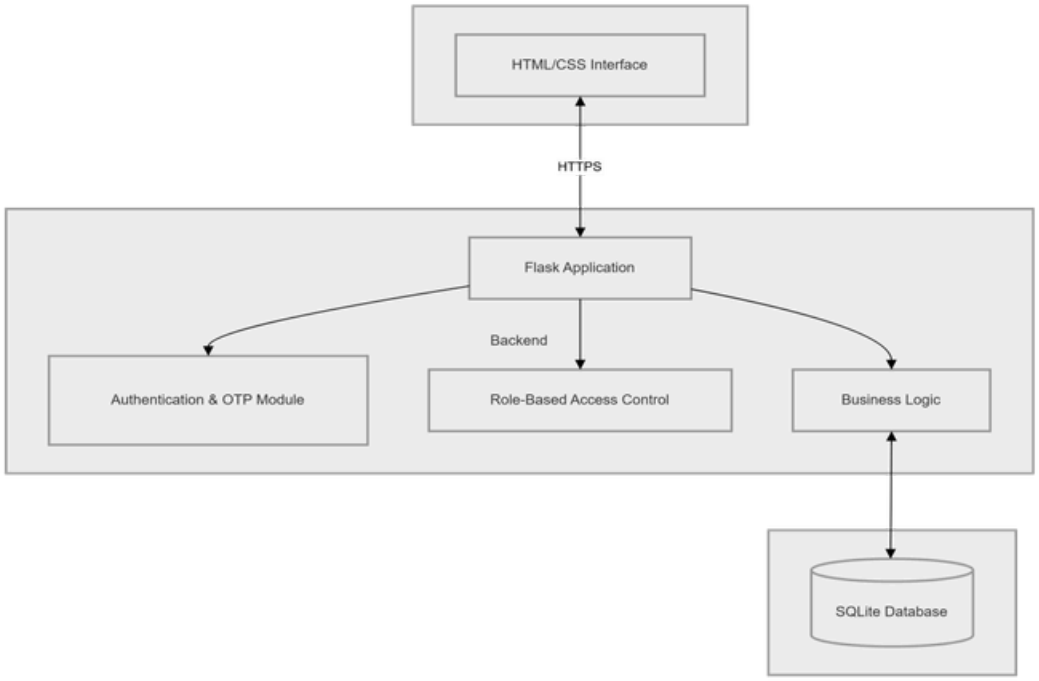
- Academic evaluators reviewing the project design
- Developers or students analyzing secure system architecture
- Cybersecurity learners focusing on identity-based protection
- Stakeholders assessing system feasibility and structure

## 1.3 System Overview

The system is designed as a **secure e-commerce platform** that integrates identity protection mechanisms into user interactions.

It follows a **client-server architecture**:

- **Frontend (HTML/CSS):** Handles user interaction and UI
- **Backend (Flask):** Manages logic, authentication, and routing
- **Database (SQLite):** Stores user, product, and order data




Thesystem ensures that only authenticated and authorized users are granted access to resources in accordance with their assigned roles, thereby maintaining strict access control.

## 2. System Design

# 2.1 Application Design

The application is designed using a **layered architecture** to maintain separation of concerns:

- **Presentation Layer:** Handles UI rendering and user input
- **Application Layer:** Processes business logic and security workflows
- **Data Layer:** Manages persistent data storage

An **email service integration** is used to send OTPs, enabling multi-factor authentication.

The design ensures:

- Modularity
- Maintainability
- Scalability for future enhancements

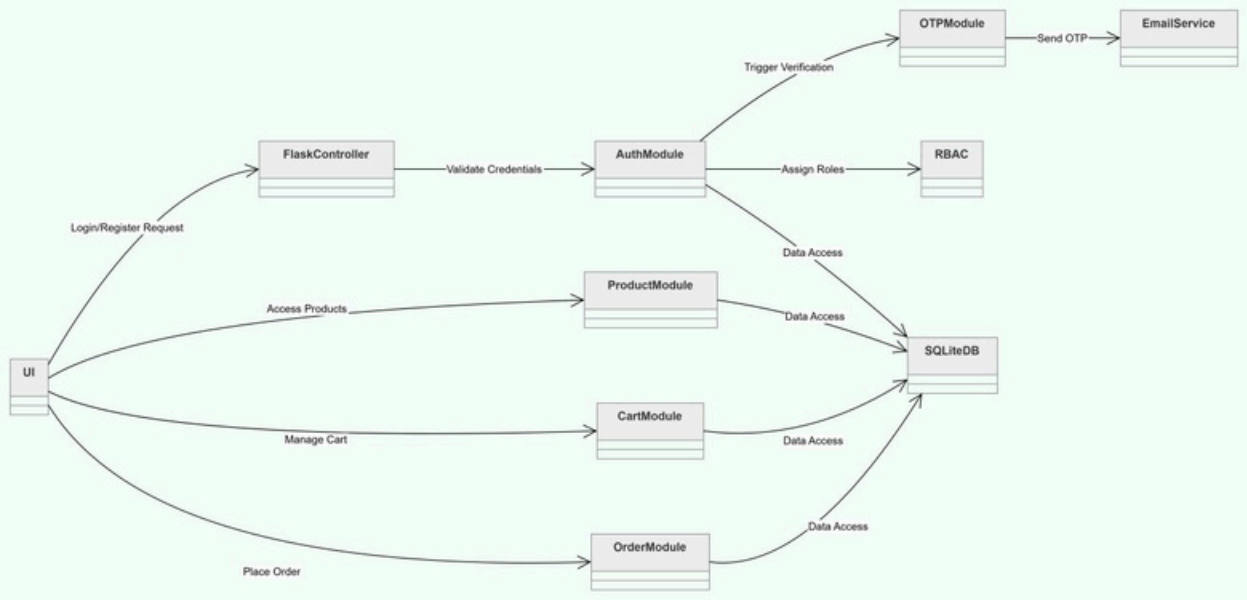



Figure 2.1 — Component Interaction Diagram

## 2.2 Process Flow

The system follows a structured and secure workflow to ensure proper identity verification and controlled access:

- Step 1 — User Registration: User submits registration details
- Step 2 — OTP Generation: System generates a one-time password
- Step 3 — OTP Verification: User verifies identity using OTP
- Step 4 — Login Request: User enters credentials
- Step 5 — Credential Validation: System verifies username and password
- Step 6 — OTP Authentication: Additional verification step using OTP
- Step 7 — Session Creation: Secure session is established
- Step 8 — Role-Based Redirection: User is redirected based on assigned role
- Step 9 — System Interaction: User performs actions such as browsing and purchasing
- Step 10 — Data Storage: Transactions and activities are securely stored

This multi-step authentication process significantly strengthens system security.

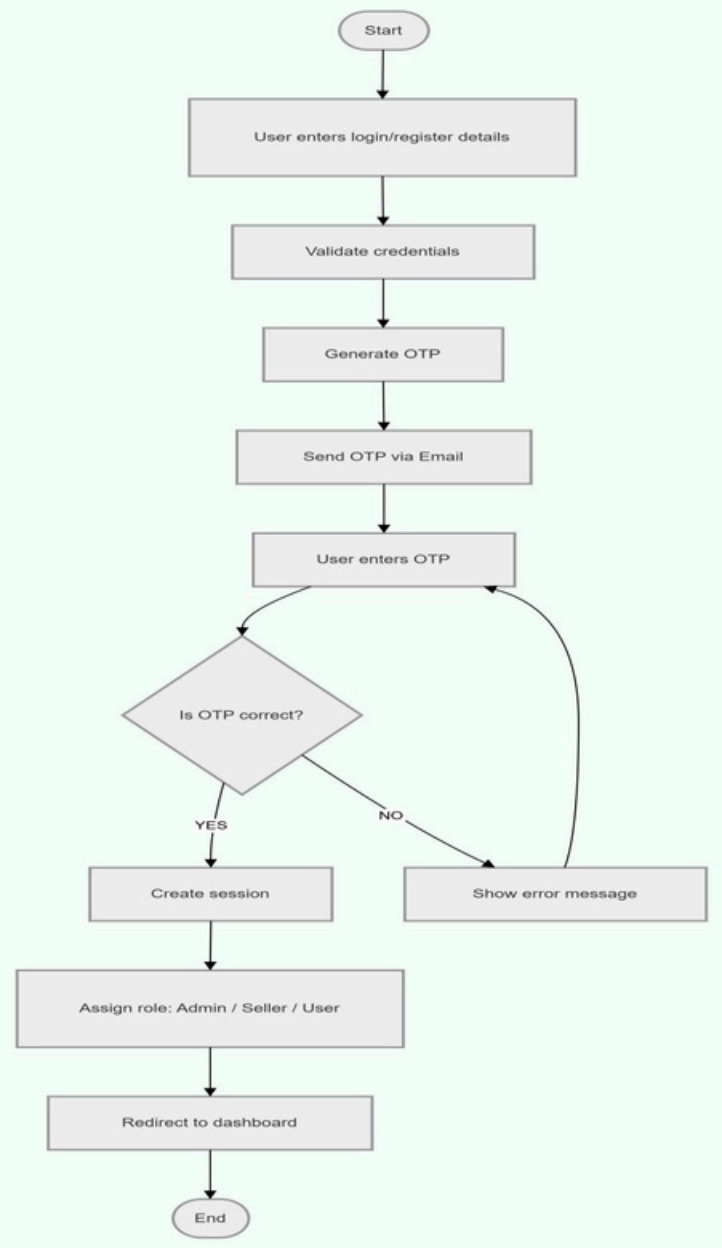



Figure 2.2 — Authentication Process Flow Diagram

### 2.3 Information Flow

The flow of information within the system is structured to ensure secure and efficient data processing:

- User inputs are transmitted from the frontend to the Flask backend
- The backend validates input data and interacts with the SQLite database
- OTPs are generated and transmitted via the email service for verification
- Verified data is securely stored within the database
- Processed responses are returned to the frontend for rendering

Session data is maintained temporarily to track user activity and ensure secure navigation across protected routes.

### 2.4 Components Design

The following table provides a detailed description of each functional component within the Identity Security Framework.

| Component | Technology | Detailed Responsibility |
|---|---|---|
| **Authentication Module** | Flask + Werkzeug | Manages /signup and /login routes. Validates credentials using check_password_hash(). Handles duplicate account prevention. Initiates OTP flow on successful password match. |
| **MFA / OTP Module** | smtplib + random | Generates a secure random 6-digit OTP. Sends via Gmail SMTP with TLS (STARTTLS, port 587). Stores OTP in Flask session server-side only. |
| **Session Manager** | Flask session | Maintains signed session cookie with keys: logged_in, user, email, role, id, otp. Signing via app.secret_key prevents client-side tampering. |
| **RBAC Engine** | Route guards | Enforces access control on every protected route using session['role'] check. Redirects unauthorized users to access_denied.html (HTTP 403 equivalent). |

| | | |
|---|---|---|
| **Password Hash Module** | Werkzeug security | Applies PBKDF2-SHA256 with random salt via generate_password_hash(). Verifies credentials via check_password_hash(). Plaintext never persisted. |
| **IAM Admin Panel** | Flask routes | Admin-only routes /admin/dashboard, /users, /promote/<id>, /demote/<id>. Provides real-time role management. Prevents self-demotion via id comparison with session. |
| **Database Layer** | sqlite3 | Opens connection per request via get_db(). Uses parameterised queries throughout. Row factory enables dict-style access. Schema auto-created by init_db(). |
| **Template Engine** | Jinja2 | Renders dynamic HTML with session variables injected by routes. Templates: login, signup, verify, dashboard, admin_dashboard, users, access_denied. |
| **Email Service** | Gmail SMTP | External integration for OTP delivery. Uses MIMEText for email composition. smtplib handles TLS handshake. App password authentication (not account password). |

## 2.5 Key Design Considerations

Strong identity verification using OTP.
Secure password storage using hashing.
Role-based access restriction.
Modular system design for scalability.
Efficient database interaction.
User-friendly interface design.

## 2.6 API Catalogue

The following table catalogues all HTTP endpoints exposed by the Flask application. All routes function as internal web endpoints following REST-like principles and primarily render HTML responses.

| Endpoint | Method(s) | Access Control | Description |
|---|---|---|---|
| / | GET | Public | Entry point —redirects to /login |
| /signup | GET, POST | Public | User registration. GET renders form. POST validates, hashes password, inserts user, redirects to /login. |
| /login | GET, POST | Public | Credential verification. GET renders form. POST checks hash, generates OTP, sends email, redirects to /verify. |
| /verify | GET, POST | Session (pending) | OTP validation. Requires session['otp'] to exist. POST sets logged_in=True and routes by role. |
| /dashboard | GET | Logged-in User | User dashboard. Redirects admin users to /admin/dashboard automatically. |
| /admin/dashboard | GET | Admin only | Admin control panel with user statistics, full user table, and IAM role management. |
| /users | GET | Admin only | Full user list with role badges and promote/demote action links. |
| /promote/<int:id> | GET | Admin only | Updates role to 'admin' for given user ID. Parameterised query prevents SQL injection. |
| /demote/<int:id> | GET | Admin only | Updates role to 'user'. Self-demotion blocked via user_id comparison with session. |
| /logout | GET | Logged-in | Calls session.clear(). Destroys all session state. Redirects to /login. |

# 3. Data Design

## 3.1 Data Model

The system uses a single SQLite database file (database.db) containing one primary table. The table design reflects the minimum necessary schema to support authentication, authorization, and role management.

| Column | Data Type | Constraint | Purpose |
|---|---|---|---|
| id | INTEGER | PRIMARY KEY, AUTOINCREMENT | Unique row identifier. Used in promote/demote URL parameters. |
| username | TEXT | NOT NULL | Login identifier. Uniqueness enforced in application logic via pre-signup check. |
| email | TEXT | NOT NULL | OTP delivery address. Displayed in dashboard. Uniqueness enforced in app logic. |
| password | TEXT | NOT NULL | Full Werkzeug hash string. Format: method$salt$hash. Never plaintext. |
| role | TEXT | DEFAULT 'user' | RBAC control field. Mutable at runtime by admin via promote/demote routes. |

## 3.2 DataAccessMechanism

- Data is accessed using structured SQLite queries through the Flask backend.
- Parameterized queries are used forsecurity.
- Flask handles all database communication.
- Data is validated before insertion.

## 3.3 Data Retention Policies

User data is retained for authentication purposes. Order data is stored for transaction tracking. Session data is temporary and cleared after logout. OTP data is removed immediately after verification.

### 3.4 Data Migration

The database schema is automatically initialized during application startup using the init_db() function, which executes CREATE TABLE IF NOT EXISTS statements. This approach eliminates the need for manual schema setup during initial deployment. To ensure data consistency, the default administrative account is created only if no existing admin record is found in the database, thereby preventing duplication during repeated application restarts.

The current implementation does not utilize any external migration frameworks (such as Alembic). As a result, any structural modifications to the database schema must be performed manually using SQL statements such as ALTER TABLE.

For future scalability, the system can be migrated from SQLite to more robust database systems such as PostgreSQL. This transition can be achieved by replacing the sqlite3 driver with an appropriate PostgreSQL adapter (e.g., psycopg2) and updating the database connection configuration. Since the application consistently uses parameterized queries, minimal changes to the query logic will be required.

## 4. Interfaces

The system exposes the following interface types for user interaction and system integration.

| Interface | Type | Description |
|---|---|---|
| **login.html** | Web UI | Username and password form. Displays error messages for invalid credentials. Link to registration page. |
| **signup.html** | Web UI | Registration form with username, email, and password fields. Inline password security guidance. Error display for duplicates. |

| | | |
|---|---|---|
| **verify.html** | Web UI | 6-digit OTP input with large monospace font field. MFA concept explanation. Error display and back-to-login link. |
| **dashboard.html** | Web UI | Authenticated user landing page showing username, email, role badge, auth method, and session status. Simulated product grid. |
| **admin_dashboard.html** | Web UI — Admin | Admin control panel with stat cards (total/admin/user counts), full user table with role badges, promote/demote actions, and RBAC concept panel. |
| **users.html** | Web UI — Admin | Dedicated user management table with role badge display and promote/demote buttons with browser confirmation dialog. |
| **access_denied.html** | Web UI — Error | HTTP 403 (Forbidden) equivalent page rendered when a user attempts to access a route above their role level. Includes RBAC education note. |
| **SQLite Interface** | Database | Direct sqlite3 connection per request. Row factory enables dict-style column access. No ORM layer. |
| **Gmail SMTP Interface** | External Email | smtplib.SMTP(smtp.gmail.com, 587) with STARTTLS. App-specific password for authentication. MIMEText for email composition. |
| **Flask Session Interface** | Cookie | HMAC-signed client-side cookie using app.secret_key. Stores authentication state and role. Accessible across all routes via the session dict. |

## 5. State and Session Management

Flask's client-side signed session cookie is used to maintain user state across requests. The session acts as the primary mechanism for enforcing both authentication and authorization within the application.

It securely stores key attributes such as user identity and role, enabling controlled access to protected routes. The session is validated on each request and is fully cleared upon logout to prevent unauthorized reuse.

## 6. Caching

The system implements minimal caching due to its lightweight and academic nature. Since the application is designed with a relatively small user base and limited data operations, complex caching mechanisms are not required. Instead, performance efficiency is achieved through optimized database queries and streamlined backend processing. Each request is handled dynamically, ensuring data consistency without relying on cached responses. This approach keeps the system simple and maintainable.

## 7. Non-Functional Requirements

### 7.1 Security Aspects

Secure password hashing mechanisms.
OTP-based multi-factor authentication.
Role-Based Access Control (RBAC).
Input validation to prevent injection attacks.
Securesession handling.
Controlled handling of user inputs and uploaded data.
Restricted access tosensitivefunctionalities.

### 7.2 Performance Aspects

The system ensures responsive user interaction through efficient request handling. Database operations are optimized to minimize latency, while the lightweight backend architecture reduces processing overhead. Session management is streamlined to support consistent performance during user activity. Overall, the system balances performance and simplicity, making it suitable for academic use while maintaining acceptable responsiveness.

# 8. References

•Flask Documentation — https://flask.palletsprojects.com/
•SQLite Documentation — https://www.sqlite.org/docs.html
•Python Official Documentation — https://docs.python.org/3/
•OWASP Top 10 Web Application Security Risks — https://owasp.org/www-project-top-ten/
•Role-Based Access Control (RBAC) Model — https://csrc.nist.gov/projects/role-based-access-control
•Web Application Security Best Practices — https://owasp.org/www-project-web-security-testing-guide/
•Flask Security Best Practices — https://flask.palletsprojects.com/en/latest/security/